Listing 4.9 An example of using delta tables to provide ACID transactions to data stored in Parquet.

```
import pandas as pd
from deltalake import DeltaTable, write_deltalake

def main():
  table_path = "./delta_demo/sensor_data"

  df1 = pd.DataFrame( #1
      {
          "sensor_id": ["temp_001", "temp_002", "humidity_001"],
          "location": ["factory", "warehouse", "factory"],
          "value": [23.5, 18.2, 65.4],
          "timestamp": pd.to_datetime(["2024-01-15 10:30:00"] * 3),
      }
  )

  write_deltalake(table_path, df1) #2
  print("Created Delta table with columns:", list(df1.columns))

  dt = DeltaTable(table_path)

  df_update = pd.DataFrame( #3
      {
          "sensor_id": ["temp_001"],
          "location": ["factory"],
          "value": [24.1],  # Updated temperature
          "timestamp": pd.to_datetime(["2024-01-15 12:00:00"]),
          "unit": ["celsius"],
      }
  )

  dt.merge( #4
      df_update,
      predicate="target.sensor_id = source.sensor_id",
      source_alias="source",
      target_alias="target",
  ).when_matched_update_all().when_not_matched_insert_all().execute()
```

#1 Define some dummy sensor data in a dataframe.
#2 Write the sensor data to a delta table, stored as parquet on the local filesystem.
#3 Update one of the attributes for sensor temp_001.
#4 Merge the updates back to the delta table.

4.5.4 Transactional data storage

Transactional databases manage reference data, configuration information, and operational records that require ACID properties and complex relational queries. These systems handle discrete updates, maintain data consistency, and support the structured data that defines digital twin entities. In contrast to analytical data stores that are designed for efficient